

LAPORAN RESMI
MODUL VI
HTML FORM DAN PHP KONEKSI
PEMROGRAMAN BERBASIS WEB DASAR



NAMA	: RAHMAD BAGUS SETIAWAN
N.R.P	: 250441100031
DOSEN	: IMAMAH. S.KOM., M. KOM
ASISTEN	: EDI PUTRA
TGL PRAKTIKUM	: 06 MEI 2026

Disetujui : APRIL 2026
Asisten

RAHMAD BAGUS SETIAWAN
25.04.411.00011



LABORATORIUM TEKNOLOGI INFORMASI
PRODI SISTEM INFORMASI
JURUSAN TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS TRUNODJOYO MADUR

DAFTAR ISI

DAFTAR ISI.....	i
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan	1
BAB II DASAR TEORI.....	2
2.1 HTML FORM DALAM PHP	2
2.1.1 Apa Itu Html <i>Form</i>	2
2.1.2 Method GET dan POST	2
2.1.3 Cara Membuat Html Form	3
2.1.4 PHP Mengolah Data <i>Form</i>	4
2.2 PHP KONEKSI DATABASE	5
2.2.1 Konsep Koneksi Database.....	5
2.2.2 Mengapa Kita Butuh Database?.....	6
2.2.3 Membuat Koneksi Database di PHP	6
2.2.4 Memisahkan Koneksi ke File Tersendiri	8
2.3 OPERASI CRUD DENGAN DATABASE	9
2.3.1 Menyimpan Data Baru ke Database (CREATE)	10
2.3.2 Menampilkan Data dari Database (READ).....	11
2.3.3 Memperbarui Data yang Ada (UPDATE).....	13
2.3.4 Menghapus Data (DELETE).....	15
2.4 KEAMANAN & AUTENTIKASI	16
2.4.1 SQL Injection dan Pencegahannya	16
2.4.2 Session dan Password Hashing	17
2.4.3 Sistem Login Sederhana.....	18
2.4.4 Memproteksi Halaman dan Role-Based Access	20
BAB III TUGAS PENDAHULUAN.....	21
BAB IV IMPLEMENTASI	24
4.1 Tugas Praktikum.....	24
4.1.1 Tugas Praktikum Soal No. 1	24
4.2 Source Code	24
4.2.1 Source Code Soal No. 1	24

4.3 Hasil.....	44
4.3.1 Hasil Soal No. 1	44
4.4 Penjelasan	45
4.4.1 Penjelasan Soal No. 1.....	45
BAB V PENUTUP.....	47
5.1 Analisa.....	47
5.2 Kesimpulan	47

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong kebutuhan akan aplikasi berbasis web yang mampu mengelola data secara dinamis dan persisten. Dalam konteks pengembangan sistem informasi, kemampuan membangun antarmuka pengguna yang terhubung langsung dengan basis data merupakan kompetensi dasar yang wajib dikuasai. *HTML Form* berperan sebagai jembatan antara pengguna dan server, memungkinkan pengiriman data secara terstruktur melalui *method GET* maupun *POST*. Di sisi server, PHP menjadi bahasa pemrograman yang memproses data tersebut sebelum berinteraksi dengan *MySQL* sebagai sistem manajemen basis data. Namun, seiring meningkatnya kompleksitas aplikasi, aspek keamanan seperti *SQL Injection* dan *XSS* tidak dapat diabaikan.

Oleh karena itu, pemahaman menyeluruh mengenai *HTML Form*, koneksi *PHP-MySQL*, operasi *CRUD*, serta mekanisme autentikasi dan otorisasi menjadi fondasi penting dalam pengembangan aplikasi web yang fungsional sekaligus aman. Implementasi *session* dan *cookie* juga menjadi bagian penting dalam menjaga keberlangsungan interaksi pengguna dengan sistem. *Session* digunakan untuk menyimpan informasi autentikasi pengguna selama proses penggunaan aplikasi berlangsung, sedangkan *cookie* membantu menyimpan data tertentu pada sisi *client* untuk meningkatkan kenyamanan dan efisiensi akses. Dengan memanfaatkan kedua mekanisme tersebut, aplikasi dapat memberikan pengalaman penggunaan yang lebih terstruktur dan aman. Penguasaan konsep integrasi antara *frontend* dan *backend* tidak hanya membantu dalam membangun aplikasi yang interaktif, tetapi juga meningkatkan kemampuan pengembang dalam merancang sistem yang efisien, stabil, mudah dipelihara, serta mampu memenuhi kebutuhan pengguna secara optimal di era digital saat ini. Seiring meningkatnya aplikasi, aspek keamanan seperti *SQL Injection* dan *XSS* tidak dapat diabaikan.

1.2 Tujuan

- Mahasiswa mampu memahami *HTML form* dan koneksi.
- Mahasiswa mampu memahami koneksi PHP dengan *database*.
- Mahasiswa mampu memahami *Create, Read, Update, dan Delete*.

BAB II

DASAR TEORI

2.1 HTML FORM DALAM PHP

2.1.1 Apa Itu Html *Form*

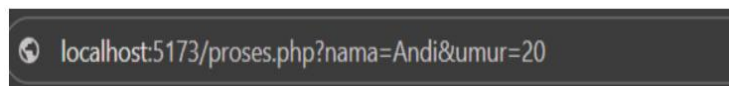
HTML *form* adalah bagian dari halaman web yang memungkinkan pengguna mengirimkan data ke server. Bayangkan *form* seperti kotak isian di halaman web. pengguna mengetikkan informasi seperti nama, alamat *email*, atau pesan, lalu menekan tombol untuk mengirimkannya. Sebelumnya kalian sudah belajar membuat tag `<form>` beserta input-input dasarnya seperti *text*, *radio*, *checkbox*, dan submit. Nah, sekarang kita fokus ke langkah berikutnya yaitu bagaimana data *form* itu diproses oleh PHP, lalu ditampilkan ke pengguna atau disimpan ke database.

2.1.2 Method GET dan POST

Setiap *form* HTML punya atribut method yang menentukan cara data dikirim ke server. Ada dua method utama: *GET* dan *POST*. Memahami perbedaannya penting karena salah pilih bisa membuat aplikasi tidak aman atau tidak berfungsi sebagaimana mestinya.

1. Method GET

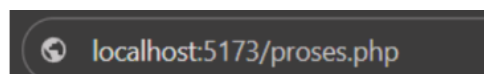
Data dikirim lewat URL, terlihat oleh siapa saja yang melihat alamat browser. Misalnya kalau *form* pakai *method="get"*, setelah *submit* URL-nya menjadi:



Data nama=Andi dan umur=20 "menempel" di *URL*. Konsekuensinya: data bisa di-*bookmark*, panjangnya terbatas (sekitar 2000 karakter), dan tidak cocok untuk informasi sensitif karena terekam di *history* browser dan *log server*.

2. Method POST

Data dikirim "tersembunyi" di *body* HTTP *request*, tidak terlihat di *URL*. Setelah submit, URL tetap bersih:



POST cocok untuk data dalam jumlah besar, data sensitif, dan operasi yang mengubah keadaan database (*insert, update, delete*). Kapan pakai yang mana?

Pakai <i>GET</i> kalau....	Pakai <i>POST</i> kalau....
Data tidak sensitif	Data sensitif (password, info pribadi)
<i>User</i> mungkin ingin <i>bookmark</i> hasilnya	Form mengubah data di database
Fitur pencarian (?cari=sepatu)	Upload file
Filter halaman (?kategori=elektronik)	<i>Login</i> dan <i>registrasi</i>

Cara mengakses datanya di PHP juga berbeda. Kalau *form* pakai *GET*, di PHP kita ambil pakai `$_GET['nama_input']`. Kalau *POST*, pakai `$_POST['nama_input']`.

2.1.3 Cara Membuat Html Form

Mari kita buat *form* sederhana untuk mengumpulkan nama, umur, dan kota pengguna:

Index.php

```
<!DOCTYPE html>
<html lang="id">
<head>
<title>Form Data Pengguna</title>
</head>
<body>
<h2>Formulir Data Pengguna</h2>
<form action="proses.php" method="POST">
<div>
<label>Nama:</label><br>
<input type="text" name="nama" required>
</div>
<br>
<div>
<label>Umur:</label><br>
<input type="number" name="umur" required>
</div>
<br>
```

```

<div>
<label>Kota:</label><br>
<select name="kota">
<option value="Jakarta">Jakarta</option>
<option value="Bandung">Bandung</option>
<option value="Surabaya">Surabaya</option>
</select>
</div>
<br><button type="submit" name="submit">Kirim Data</button>
</form>
</body>
</html>

```

- Beberapa hal yang perlu diperhatikan: `action="proses.php"` file tujuan yang akan menerima dan memproses data.
- `method="POST"` cara pengiriman, sesuai diskusi sebelumnya.
- Atribut `name` di setiap input adalah kunci penting. Nilai inilah yang nanti dipanggil di PHP via `$_POST['nama']`. Tanpa atribut `name`, data input tidak akan terkirim ke server.
- `required` atribut HTML5 yang mencegah form di-submit kalau *field* kosong. Ini validasi sederhana di sisi browser.

2.1.4 PHP Mengolah Data Form

Setelah pengguna menekan tombol "Kirim Data", data form dikirim ke `proses.php`. Sekarang giliran PHP yang menanganinya. Berikut isi file `proses.php`:

proses.php

```

<?php
// Cek apakah halaman ini diakses lewat submit form (POST)
if ($_SERVER["REQUEST_METHOD"] == "POST") {
// Ambil data dari form, bersihkan untuk tampilan (anti-XSS)
$nama = htmlspecialchars($_POST['nama']);
$umur = htmlspecialchars($_POST['umur']);
$kota = htmlspecialchars($_POST['kota']);
// Tampilkan hasil ke pengguna
echo "<h2>Konfirmasi Data</h2>";
echo "Terima kasih, <b>$nama</b>.<br>";
echo "Kamu yang berumur $umur tahun tinggal di kota $kota.";
echo "<br><br><a href='index.php'>Kembali ke Form</a>";
} else {
// Kalau seseorang buka proses.php langsung tanpa lewat form
echo "Silakan isi form terlebih dahulu!";
}

```

Penjelasan baris demi baris:

1. `$_SERVER["REQUEST_METHOD"] == "POST"`: kita cek dulu apakah file ini diakses karena ada *form* yang di-*submit*, atau seseorang mengetik URL `proses.php` langsung di *browser*. Kalau diakses langsung, tampilkan pesan agar mereka isi *form* dulu. Ini praktik kecil tapi penting untuk mencegah *error*.
2. `$_POST['nama']`: cara mengambil nilai dari *input* yang punya `name="nama"`. Pola ini berlaku untuk semua *input* dengan method *POST*. Untuk method *GET*, ganti `$_POST` jadi `$_GET`.
3. `htmlspecialchars(...)`: fungsi keamanan yang mengubah karakter HTML berbahaya seperti `<`, `>`, `"`, dan `&` menjadi versi "aman" (`<`, `>`, `"`, `&`). Ini melindungi halaman dari serangan XSS (*Cross-Site Scripting*), kalau ada pengguna nakal yang mencoba memasukkan `<script>alert('hacked')</script>` sebagai nama, fungsi ini akan menampilkannya sebagai teks biasa, bukan menjalankannya sebagai *script*. Apa yang terjadi ketika *form* di-*submit*? Saat pengguna mengklik tombol "Kirim Data":
 1. *Browser* mengumpulkan semua nilai input yang punya atribut `name`.
 2. *Browser* mengirim data tersebut ke URL yang ditentukan di *action* (yaitu `proses.php`), dengan *method* yang ditentukan di *method* (*POST*).
 3. Server menjalankan `proses.php`. Di file ini, PHP punya akses ke data tadi lewat *array* global `$_POST`.
 4. PHP mengolah data dan mengirim balik HTML ke browser untuk ditampilkan.

2.2 PHP KONEKSI DATABASE

2.2.1 Konsep Koneksi Database

Koneksi *database* adalah cara bagi program PHP untuk "berbicara" dengan *database*. Bayangkan *database* seperti rak buku besar yang berisi banyak informasi tersusun rapi. Kita tidak bisa langsung mengambil buku dari rak, kita butuh seorang petugas perpustakaan sebagai perantara. Dalam analogi ini, PHP adalah petugas perpustakaan yang menerima permintaan dari halaman web, mengambil atau

menyimpan data ke *database*, lalu mengembalikan hasilnya ke pengguna. Alur kerjanya sederhana:

1. Pengguna membuka halaman web atau mengisi *form*.
 2. PHP menerima permintaan dan membuka koneksi ke *database*.
 3. PHP mengirim "pertanyaan" (*query*) ke *database* misalnya "tampilkan semua data guru".
 4. *Database* menjalankan *query* dan mengembalikan hasilnya ke PHP.
 5. PHP menyajikan hasil tersebut sebagai HTML ke browser pengguna.
- Tanpa koneksi, PHP dan *database* hanyalah dua sistem terpisah yang tidak saling kenal.

2.2.2 Mengapa Kita Butuh Database?

Sebelumnya kalian sudah belajar menyimpan data di variabel dan *array* PHP. Tapi data di sana punya kelemahan besar: data hilang saat halaman selesai dimuat atau server di-*restart*. Coba bayangkan kalau aplikasi *e-commerce* menyimpan daftar produk di *array* PHP saja, setiap kali server *restart*, semua produk hilang. Setiap kali pengguna *refresh* halaman, data *form* yang baru diisi lenyap. *Database* memecahkan masalah ini, ia menyimpan data secara persisten (tidak hilang sampai sengaja dihapus), bisa diakses oleh banyak pengguna sekaligus, dan punya mekanisme untuk mencari, memfilter, serta mengelola data dalam jumlah besar dengan efisien. Kita akan menggunakan *MySQL* sebagai *database*, dan *mysqli* sebagai *library* PHP yang menghubungkan keduanya.

2.2.3 Membuat Koneksi Database di PHP

Pertama-tama, pastikan *XAMPP* sudah berjalan *Apache* dan *MySQL* aktif di *Control Panel*. Kalau kalian pakai Laragon sebagai alternatif, prinsipnya sama: aktifkan *Apache* dan *MySQL* dari panel Laragon. Buka *phpMyAdmin* di browser (<http://localhost/phpmyadmin>) untuk memastikan *database server* bisa diakses. Berikut kode untuk membuat koneksi PHP ke *MySQL*. Simpan sebagai file bernama koneksi.php: **koneksi.php**

```
<?php
// Informasi server database
$servername = "localhost"; // alamat server (default)
$username = "root"; // username default
$password = ""; // password kosong (default)
```

```

$database = "db_modul6"; // nama database yang akan kita
pakai
// Membuat koneksi
$conn = new mysqli($servername, $username, $password,
$database);
// Mengecek koneksi
if ($conn->connect_error) {
die("Koneksi ke database gagal: " . $conn->connect_error);
}
// Kalau sampai ke baris ini, koneksi berhasil
// echo "Koneksi berhasil!"; // hapus tanda komentar untuk
testing
?>

```

Penjelasan:

1. Empat variabel di atas adalah informasi yang dibutuhkan untuk *login* ke *database* server: server-nya di mana, *login* pakai *username* apa, *password*-nya apa, dan *database* mana yang ingin diakses. Untuk *XAMPP* yang baru di-install, default-nya selalu *username* root dengan *password* kosong ("").
2. `new mysqli(...)` membuat objek koneksi baru. Kalau berhasil, objek `$conn` inilah yang nanti dipakai untuk semua operasi *database* (*insert*, *select*, *update*, *delete*).
3. `$conn->connect_error` mengecek apakah ada *error* saat koneksi. Kalau ada (misalnya *MySQL* belum dijalankan, atau nama *database* salah), fungsi `die()` akan menghentikan eksekusi dan menampilkan pesan *error*. Tanpa pengecekan ini, *error* koneksi menyebabkan halaman blank tanpa informasi apa-apa sangat menyulitkan saat debugging.
4. Baris `echo "Koneksi berhasil!"` sengaja dikomentari. Aktifkan sementara saat testing pertama kali untuk memastikan koneksi jalan, lalu komentari lagi. Kalau dibiarkan aktif, pesan ini akan muncul di setiap halaman yang memakai koneksi dan mengganggu tampilan.

TIPS: Sebelum menjalankan kode ini, pastikan database `db_modul6` sudah dibuat di phpMyAdmin. Pembuatan database dan tabelnya akan dibahas dibawah

2.2.4 Memisahkan Koneksi ke File Tersendiri

Kalau setiap kali butuh koneksi database harus menulis ulang semua kode di atas, akan muncul tiga masalah:

- 1) Kode jadi panjang dan berulang. Bayangkan kalau aplikasi/sistem punya 20 file harus copy-paste 20 kali.
- 2) Kalau ada perubahan, repot. Misalnya password database diubah, kita harus mencari dan mengubah di 20 file. Lupa satu error di mana-mana.
- 3) Rentan typo. Setiap kali ditulis ulang, peluang typo bertambah. PHP punya solusi: `include`. Fungsi ini menyisipkan isi file lain ke posisi tempat kita memanggilnya seolah-olah kode dari file itu ditulis langsung di sana.

1. `include` vs `require` vs `require_once`

PHP sebenarnya punya beberapa varian fungsi untuk menyertakan file:

Fungsi	Perilaku saat file tidak ditemukan
<code>include 'file.php'</code>	Menampilkan warning, kode di bawahnya tetap dijalankan
<code>require 'file.php'</code>	Menampilkan fatal error, eksekusi dihentikan total
<code>include_once 'file.php'</code>	Sama seperti <code>include</code> , tapi mencegah file disertakan dua kali
<code>require_once 'file.php'</code>	Sama seperti <code>require</code> , tapi mencegah file disertakan dua kali

Untuk file koneksi database, sebenarnya `require_once` lebih aman. Alasannya:

- a) kalau *file* koneksi hilang, lebih baik aplikasi langsung berhenti daripada lanjut tanpa *database* (yang akan menyebabkan error berantai)
- b) *once* mencegah *error* kalau tidak sengaja file disertakan berkali-kali. Kali ini kita tetap pakai *include* agar konsisten dan lebih sederhana untuk pemula. Saat kalian membangun aplikasi/sistem nyata nanti, silakan langsung pakai *require_once*.

2. Struktur Folder yang Akan Kita Pakai

htdocs/	
└─ modul6/	
├─ koneksi.php	← file koneksi (dibuat sekali, dipakai semua)
├─ index.php	← form input data
├─ proses.php	← memproses input dari form
├─ tampil_data.php	← menampilkan data dari database
├─ update.php	← memperbarui data
└─ hapus.php	← menghapus data

Folder *htdocs* adalah lokasi default XAMPP (biasanya C:\xampp\htdocs). Kalau pakai Laragon, lokasinya di C:\laragon\www. Sesuaikan saja. Setiap file PHP yang butuh akses *database* cukup dimulai dengan satu baris:

```
<?php
include 'koneksi.php';
// ... kode operasi database di sini
?>
```

2.3 OPERASI CRUD DENGAN DATABASE

Sebelum mulai, kita perlu menyiapkan *database* dan tabel di MySQL. Buka phpMyAdmin (<http://localhost/phpmyadmin>), klik tab SQL, lalu jalankan perintah berikut:

```
-- Membuat database baru
CREATE DATABASE IF NOT EXISTS db_modul6;
USE db_modul6;

-- Membuat tabel guru
CREATE TABLE guru (
id INT AUTO_INCREMENT PRIMARY KEY,
nama VARCHAR(100) NOT NULL,
umur INT NOT NULL
);

-- Tambahkan beberapa data awal untuk testing
INSERT INTO guru (nama, umur) VALUES
('Andi', 35),
('Budi', 42),
('Citra', 28);
```

Penjelasan:

- *CREATE DATABASE IF NOT EXISTS* membuat *database* hanya kalau belum ada aman dijalankan berulang.

- *USE db_modul6* memilih database yang akan dipakai. Kolom id pakai *AUTO_INCREMENT PRIMARY KEY* artinya *MySQL*
- otomatis memberikan nomor unik untuk tiap baris baru. Kita tidak perlu mengisi id saat *insert*.
- Tiga data awal berfungsi supaya operasi *Read* langsung menampilkan sesuatu, tidak kosong.

2.3.1 Menyimpan Data Baru ke Database (CREATE)

Operasi *Create* adalah cara menambahkan data baru ke dalam *database*. Misalnya: pengguna mengisi *form*, lalu kita simpan ke tabel. Kita sudah membuat *index.php* (*form*) dan *proses.php* (yang hanya menampilkan kembali data). Sekarang kita upgrade *proses.php* supaya juga menyimpan data ke *database*.

Index.php

```
<!DOCTYPE html>
<html lang="id">
<head>
<title>Tambah Data Guru</title>
</head>
<body>
<h2>Form Tambah Data Guru</h2>
<form action="proses.php" method="POST">
<label>Nama:</label><br>
<input type="text" name="nama" required><br><br>
<label>Umur:</label><br>
<input type="number" name="umur" required><br><br>
<button type="submit" name="submit">Simpan</button>
</form>
<br>
<a href="tampil_data.php">Lihat Daftar Guru</a>
</body>
</html>
```

proses.php

```
<?php
// 1. Panggil koneksi database
include 'koneksi.php';
// 2. Cek apakah form sudah di-submit
if (isset($_POST['submit'])) {
// 3. Ambil data dari form
$nama = $_POST['nama'];
$umur = (int) $_POST['umur']; // dipaksa jadi integer
```

```
// 4. Buat query INSERT
$sql = "INSERT INTO guru (nama, umur) VALUES ('$nama',
'$umur')";
// 5. Jalankan query dan cek hasilnya
if ($conn->query($sql) === TRUE) {
echo "<h3>Berhasil!</h3>";
echo "Data guru bernama <b>$nama</b> sudah masuk ke
database.";
echo "<br><br><a href='tampil_data.php'>Lihat Daftar
Guru</a>";
echo " | <a href='index.php'>Tambah Data Lagi</a>";
} else {
echo "Gagal menyimpan: " . $conn->error;
}
// 6. Tutup koneksi
$conn->close();
}
?>
```

Penjelasan langkah demi langkah:

1. `include 'koneksi.php'` menyertakan file koneksi yang dibuat sebelumnya. Setelah baris ini, variabel `$conn` siap digunakan.
2. `isset($_POST['submit'])` cek apakah tombol submit *form* sudah ditekan. Mencegah eksekusi kalau file dibuka langsung tanpa *form*.
3. `(int) $_POST['umur']` paksa konversi ke *integer*. Praktik kecil tapi penting: kalau user iseng kirim teks di field umur (lewat *tools* developer), `(int)` mengubahnya jadi 0, mencegah *error SQL*.
4. `INSERT INTO ... VALUES (...)` perintah SQL untuk menambahkan baris baru. Variabel PHP disisipkan langsung ke *string* SQL.
5. `$conn->query($sql)` eksekusi *query*. Mengembalikan `TRUE` kalau berhasil, `FALSE` kalau gagal.
6. `$conn->close()` menutup koneksi setelah selesai. Praktik baik untuk membebaskan *resource* server.

2.3.2 Menampilkan Data dari Database (READ)

Operasi *Read* adalah cara mengambil data dari *database* dan menampilkannya. Biasanya berupa daftar (list) yang ditampilkan dalam tabel HTML.

tampil_data_php

```

<?php
// 1. Panggil koneksi
include 'koneksi.php';
// 2. Query untuk mengambil semua data dari tabel guru
$sql = "SELECT * FROM guru";
$result = $conn->query($sql);
?>
<!DOCTYPE html>
<html>
<head>
<title>Daftar Guru</title>
<style>
table { border-collapse: collapse; width: 60%; }

th, td { border: 1px solid black; padding: 8px; text-align: left; }

th { background-color: #f2f2f2; }
.aksi a { margin-right: 8px; }
</style>
</head>
<body>
<h2>Daftar Guru</h2>
<a href="index.php">+ Tambah Guru Baru</a><br><br>
<?php if ($result->num_rows > 0): ?>
<table><tr>
<th>ID</th>
<th>Nama</th>
<th>Umur</th>
<th>Aksi</th>
</tr>
<?php
// 3. Looping setiap baris data
while ($row = $result->fetch_assoc()):
?>
<tr>
<td><?= $row['id']; ?></td>
<td><?= $row['nama']; ?></td>
<td><?= $row['umur']; ?></td>
<td class="aksi">
<a href="update.php?id=<?= $row['id'];
?>">Edit</a>

<a href="hapus.php?id=<?= $row['id']; ?>"
onclick="return confirm('Yakin mau hapus

```

```

data ini?')">Hapus</a>
</td>
</tr>
<?php endwhile; ?>
</table>
<?php else: ?>
<p>Tidak ada data guru.</p>
<?php endif; ?>
<?php $conn->close(); ?>
</body>
</html>

```

Penjelasan poin penting:

1. *SELECT * FROM* guru perintah *SQL* mengambil semua kolom dan semua baris dari tabel guru.
2. `$result->num_rows` properti yang berisi jumlah baris hasil *query*. Kita cek dulu sebelum mencoba menampilkan, agar kalau kosong tidak *error*.
3. `$result->fetch_assoc()` mengambil satu baris dari hasil query sebagai *array* asosiatif. Dipakai dalam *loop while*, *looping* berhenti otomatis saat tidak ada baris lagi.
4. `<?= ... ?>` sintaks pendek untuk `<?php echo ... ?>`. Lebih ringkas saat dipakai di tengah HTML.
5. Kolom Aksi dengan tombol Edit dan Hapus, link membawa parameter id di URL (`update.php?id=3`). Inilah yang akan dipakai oleh `update.php` dan `hapus.php` untuk tahu data mana yang harus diolah.
6. `onclick="return confirm(...)"` JavaScript bawaan browser untuk dialog konfirmasi. Kalau user klik "*Cancel*", link tidak akan dijalankan.

2.3.3 Memperbarui Data yang Ada (UPDATE)

Operasi *Update* memperbarui data yang sudah ada di *database*. Polanya: user klik link "Edit" di daftar → muncul form yang sudah berisi data lama → user ubah → submit → data di database berubah. Kita pakai satu file `update.php` yang menangani dua kasus:

- Saat dibuka via *GET* (`update.php?id=3`) → tampilkan *form* berisi data lama
- Saat di-submit via *POST* → simpan perubahan ke database

update.php

```
<?php
include 'koneksi.php';
// === BAGIAN 1: Saat form di-submit (POST) ===
if (isset($_POST['update'])) {
    $id = (int) $_POST['id'];
    $nama = $_POST['nama'];
    $umur = (int) $_POST['umur'];
    $sql = "UPDATE guru SET nama='$nama', umur='$umur' WHERE
    id='$id'";
    if ($conn->query($sql) === TRUE) {
        echo "<h3>Berhasil!</h3>";
        echo "Data guru dengan ID $id berhasil diperbarui.";
        echo "<br><a href='tampil_data.php'>Lihat Daftar</a>";
    } else {
        echo "Gagal memperbarui: " . $conn->error;
    }
    $conn->close();
    exit; // hentikan eksekusi, jangan lanjut ke form di bawah}
// === BAGIAN 2: Saat dibuka via GET (tampilkan form pre-filled)
===
if (!isset($_GET['id'])) {
    die("ID tidak ditemukan!");
}
$id = (int) $_GET['id'];
$sql = "SELECT * FROM guru WHERE id='$id'";
$result = $conn->query($sql);
if ($result->num_rows == 0) {
    die("Data tidak ditemukan!");
}
$row = $result->fetch_assoc();
?>
<!DOCTYPE html>
<html>
<head><title>Edit Data Guru</title></head>
<body>
<h2>Edit Data Guru</h2>
<form action="update.php" method="POST">
<input type="hidden" name="id" value="<?= $row['id'];
?>">
<label>Nama:</label><br>
<input type="text" name="nama" value="<?= $row['nama'];
?>" required><br><br>
<label>Umur:</label><br>
<input type="number" name="umur" value="<?=
$row['umur']; ?>" required><br><br>
```

```

<button type="submit" name="update">Simpan
Perubahan</button>
<a href="tampil_data.php">Batal</a>
</form>
</body>
</html>

```

Penjelasan poin kunci:

1. Dua "bagian" dalam satu *file*, dibedakan oleh `isset($_POST['update'])` (cek apakah *form* di-submit). Ini pola umum di PHP yang menghemat *file*.
2. `exit` setelah bagian *POST* penting! Tanpa ini, setelah update sukses, *PHP* akan lanjut ke bagian *GET* dan menampilkan *form* lagi.
3. `<input type="hidden" name="id">` input tersembunyi yang membawa ikut ter-submit, supaya bagian *POST* tahu data mana yang harus di-update.
4. `value="<?= $row['nama']; ?>"` mengisi otomatis *form* dengan data lama dari *database*. Inilah yang membuat *user* melihat data lama, bukan *form* kosong.
5. *WHERE* `id='$id'` di query *UPDATE* WAJIB. Tanpa *WHERE*, semua baris di tabel akan ter-update. Selalu pakai *WHERE* saat update data spesifik.

2.3.4 Menghapus Data (DELETE)

Operasi *Delete* menghapus baris dari *database*. Polanya: *user* klik link "Hapus" di daftar (yang membawa id di URL) → konfirmasi → data dihapus.

hapus.php

```

<?php
include 'koneksi.php';
// 1. Cek apakah parameter id ada di URL
if (!isset($_GET['id'])) {
die("ID tidak ditemukan!");
}
// 2. Ambil id dari URL
$id = (int) $_GET['id'];
// 3. Jalankan query DELETE
$sql = "DELETE FROM guru WHERE id='$id'";
if ($conn->query($sql) === TRUE) {
// 4. Setelah berhasil, langsung redirect kembali ke daftar
header("Location: tampil_data.php");
exit;
} else {echo "Gagal menghapus: " . $conn->error;
}
}

```

```
$conn->close();  
?>
```

Penjelasan:

1. Cek `$_GET['id']` kalau seseorang membuka `hapus.php` langsung tanpa parameter, hentikan dengan pesan *error*.
2. `(int) $_GET['id']` paksa jadi *integer*. Ini juga bentuk pertahanan kecil terhadap input nakal di *URL*.
3. *DELETE FROM guru WHERE id='\$id'* perintah SQL menghapus baris. *WHERE* wajib tanpa itu, seluruh tabel akan terhapus.
4. *header("Location: ...") redirect browser* ke halaman lain. Setelah *delete* sukses, langsung kembalikan *user* ke daftar agar mereka langsung melihat data yang hilang.
5. *exit* setelah *header()* *best practice*. Tanpa ini, kode di bawah masih dijalankan walaupun browser sudah dialihkan.

2.4 KEAMANAN & AUTENTIKASI

Sampai di sini, kalian sudah berhasil membuat aplikasi CRUD yang berfungsi. Tapi kode kalian sebenarnya masih punya dua kelemahan besar: rentan terhadap serangan keamanan dan belum punya sistem *login*. Bagian ini menyelesaikan keduanya.

2.4.1 SQL Injection dan Pencegahannya

SQL Injection adalah serangan di mana penyerang menyisipkan perintah SQL berbahaya melalui input pengguna. Penyebabnya: kode kita menyisipkan input langsung ke *string* SQL

```
$sql = "DELETE FROM guru WHERE id='$id'";  
}
```

Selama `$id` berisi angka wajar, *query* baik-baik saja. Tapi penyerang bisa memanipulasi *URL* menjadi `hapus.php?id=' OR '1'=1`. Setelah PHP menggabungkan, *query* menjadi *DELETE FROM guru WHERE id=" OR '1'=1'*. Karena `'1'='1'` selalu *TRUE*, seluruh data di tabel *guru* terhapus. Solusinya: *prepared statement*. Cara ini memisahkan struktur *query* dari data *input*. Kita kirim "template" *query* dengan *placeholder* `?` ke MySQL, lalu mengisi data ke *placeholder* secara terpisah. MySQL memperlakukan input sebagai data, bukan

sintaks SQL. Tiga langkah utama: prepare → bind → execute. Berikut versi aman dari proses.php (*CREATE*)

```
<?php
include 'koneksi.php';
if (isset($_POST['submit'])) {
    $nama = $_POST['nama'];
    $umur = (int) $_POST['umur'];
    // Versi prepared statement
    $stmt = $conn->prepare("INSERT INTO guru (nama, umur) VALUES
    (?, ?)");
    $stmt->bind_param("si", $nama, $umur);
    if ($stmt->execute()) {
        echo "Data berhasil disimpan!";
    } else {
        echo "Gagal: " . $stmt->error;
    }
    $stmt->close();
    $conn->close();
}
?>
```

:Pada `bind_param("si", ...)`, karakter `s` artinya string dan `i` artinya integer. Pola yang sama berlaku untuk operasi lain *UPDATE*, *DELETE*, dan *SELECT WHERE*. Khusus untuk *SELECT*, gunakan `$stmt->get_result()` setelah *execute* untuk mendapat objek hasil:

```
// SELECT WHERE
$stmt = $conn->prepare("SELECT * FROM guru WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
$result = $stmt->get_result();
$row = $result->fetch_assoc();

// DELETE
$stmt = $conn->prepare("DELETE FROM guru WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
```

2.4.2 Session dan Password Hashing

Session adalah cara PHP "mengingat" data antar *request* untuk pengguna yang sama. Saat *user login*, kita simpan informasinya di *session*, lalu setiap *request* berikutnya PHP mengenali user tanpa perlu *login* ulang. Empat perintah utama:

```
session_start(); // Mulai/lanjutkan session – WAJIB
```

```

di awal file
$_SESSION['user_id'] = 7; // Simpan data ke session
echo $_SESSION['user_id']; // Baca data dari session
session_destroy(); // Hancurkan session (logout)

```

Password Hashing. Password tidak boleh disimpan dalam bentuk asli di database. Kalau database bocor, semua password kebaca. Solusinya: hash password menjadi string acak yang tidak bisa dibalik. Kolom database untuk menyimpan hash harus VARCHAR(255) karena hash bisa panjang dan formatnya bisa berubah di versi PHP berikutnya. PHP punya dua fungsi untuk ini:

```

// Saat REGISTER – simpan hash
$hash = password_hash("rahasia123", PASSWORD_DEFAULT);
// Saat LOGIN – verifikasi
if (password_verify($password_input, $hash_dari_database)) {
// Password benar
}

```

2.4.3 Sistem Login Sederhana

Sekarang kita gabungkan semuanya. Mulai dari struktur tabel:

```

CREATE TABLE users (
id INT AUTO_INCREMENT PRIMARY KEY,
username VARCHAR(50) UNIQUE NOT NULL,
password VARCHAR(255) NOT NULL,
role ENUM('admin', 'user') DEFAULT 'user'
);

```

File: register.php saat user mendaftar, password di-hash dulu sebelum disimpan. User baru otomatis dapat role user. Untuk membuat akun admin, ubah role secara manual lewat phpMyAdmin:

```

<?php
include 'koneksi.php';
if (isset($_POST['daftar'])) {
$username = $_POST['username'];
$hash = password_hash($_POST['password'], PASSWORD_DEFAULT);
$stmt = $conn->prepare("INSERT INTO users (username,
password) VALUES (?, ?)");
$stmt->bind_param("ss", $username, $hash);
$stmt->execute();
echo "Registrasi berhasil! <a href='login.php'>Login</a>";
}
?>

```

```
<form method="POST">
Username: <input type="text" name="username" required><br>
Password: <input type="password" name="password"
required><br>
<button type="submit" name="daftar">Daftar</button>
</form>
```

File: login.php verifikasi password lalu simpan info ke *session*. Pesan *error* sengaja generik ("*Username* atau *password* salah"), bukan spesifik seperti "*username* tidak ditemukan" agar penyerang tidak tahu *username* mana yang valid:

```
<?php
session_start();
include 'koneksi.php';
if (isset($_POST['login'])) {
$stmt = $conn->prepare("SELECT * FROM users WHERE
username=?");
$stmt->bind_param("s", $_POST['username']);
$stmt->execute();
$user = $stmt->get_result()->fetch_assoc();
if ($user && password_verify($_POST['password'],
$user['password'])) {
$_SESSION['user_id'] = $user['id']; $_SESSION['username'] =
$user['username'];
$_SESSION['role'] = $user['role'];
header("Location: tampil_data.php");
exit;
}
echo "Username atau password salah.";
}
?>
<form method="POST">
Username: <input type="text" name="username" required><br>
Password: <input type="password" name="password"
required><br>
<button type="submit" name="login">Login</button>
</form>
```

File: *logout.php*:

```
<?php
session_start();
session_destroy();
header("Location: login.php");
exit;
?>
```

2.4.4 Memproteksi Halaman dan Role-Based Access

Setelah punya sistem *login*, halaman seperti `tampil_data.php` perlu diproteksi agar hanya user yang sudah *login* yang bisa akses. Daripada *copy-paste* pengecekan di setiap file, buat satu file helper dan *include* di mana perlu: `auth.php`

```
<?php
session_start();
// Cek apakah user sudah login
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}
// Fungsi untuk halaman khusus admin
function wajib_admin() {
    if ($_SESSION['role'] !== 'admin') {}
}
?>
```

Cara pakai sangat sederhana, tambahkan satu baris di paling atas file yang ingin diproteksi contoh:

```
<?php
include 'auth.php'; // <- cek login dulu
include 'koneksi.php';
// ... kode halaman seperti biasa ...
?>
```

Selain proteksi *server-side* di atas, kita juga bisa menyembunyikan tombol/link tertentu dari user biasa di tampilan. Misalnya, di `tampil_data.php`, link "Hapus" hanya muncul untuk admin:

```
<?php if ($_SESSION['role'] === 'admin'): ?>
<a href="hapus.php?id=<?= $row['id']; ?>">Hapus</a>
<?php endif; ?>
```

TIPS: Selalu gunakan dua lapis proteksi, sembunyikan UI untuk kenyamanan pengguna, dan validasi di server untuk keamanan. Jangan andalkan UI saja, karena penyerang bisa tetap mengakses URL `hapus.php?id=X` langsung lewat browser.

10/05/24

BAB III
TUGAS PENDAHULUAN

3.1 SOAL

1. Jelaskan perbedaan antara method GET dan POST pada form HTML. Berikan satu contoh kasus penggunaan yang lebih cocok untuk masing-masing method tersebut
2. Apa yang dimaksud dengan serangan XSS (cross-site scripting) dan bagaimana fungsi PHP `htmlspecialchars()` dalam PHP membantu mencegahnya? Jelaskan pula mengapa fungsi ini tidak dapat digunakan untuk mencegah serangan SQL Injection
3. Saat sebuah form HTML di-submit ke file PHP untuk disimpan ke database, jelaskan secara berurutan langkah-langkah yang dilakukan PHP, mulai dari menerima data form hingga data tersimpan ke database. Sebutkan minimal lima langkah utama.
4. Pada operasi Delete dalam aplikasi web. ID data yang ingin dihapus biasanya dikirimkan melakukan URL (contoh: `hapus.php?id=5`). Jelaskan alasan umum mengapa pengiriman ID untuk operasi Delete menggunakan method GET. Sebutkan juga minimal dua hal yang perlu dilakukan di sisi PHP saat menerima parameter ID dari URL agar tidak terjadi error atau masalah saat data diproses
5. Jelaskan perbedaan antara fungsi `include`, `require`, `include_once` dan `require_once` di PHP. Mengapa untuk file koneksi database lebih disarankan menggunakan `require_once` dibandingkan `include`

3.2 JAWABAN

1. Perbedaan Get dan Post
 - Get
 - a. Data dikirim lewat URL (`?nama = nilai`)

[Signature]

13/05
20

- b. bisa dilihat oleh user, panjang terbatas
- c. Cocok untuk pencarian / filter data
contoh: search artikel
- Post
 - a. Data dikirim lewat body (tidak terlihat URL)
 - b. lebih aman dan bisa kirim data user
 - c. Cocok untuk: Form login
contoh: Form Registrasi
- 2. • XSS adalah serangan dengan menyisipkan script ke input user agar dijalankan di browser korban
 - Fungsi htmlspecialchars() adalah mengubah karakter khusus menjadi aman, agar script tidak di eksekusi browser
 - Kenapa tidak mencegah SQL Injection?
 - a. karena XSS menyerang browser (output HTML)
 - b. Sedangkan SQL Injection menyerang database (query SQL)
 - c. Pencegah SQL Injection adalah prepared statement
- 3. Langkah PHP menyimpan data ke database
 - 1. Form di-submit
 - 2. PHP menerima data
 - 3. Validasi dan Sanitasi data
 - 4. Koneksi ke database
 - 5. Menyusun query SQL (INSERT)
 - 6. Eksekusi query
 - 7. Cek berhasil / gagal
 - 8. Tutup koneksi / tampilkan hasil
- 4. Akses pakai GET
 - Mudah disanggil lewat link
 - Praktis untuk aksi cepat (tombol hapus)
- Hal yang harus dilakukan di PHP
 - 1) Cek apakah id ada
`if (isset($_GET['id']))`
 - 2) Validasi id (angka)
`$id = (int) $_GET['id'];`


19/05
26

5. • Include
Jika file error maka lanjut jalan
- Require
Jika file error maka program berhenti
 - Include_once
Jika melakukan include tapi hanya sekali
 - Require_once
Require tapi hanya sekali
- Mengapa pakai require_once untuk koneksi database
- koneksi database itu penting, maka kalau gagal harus berhenti
 - Mencegah file koneksi dimuat berkali-kali (error koneksi ganda)

BAB IV IMPLEMENTASI

4.1 Tugas Praktikum

4.1.1 Tugas Praktikum Soal No. 1

- *Database* minimal 2 tabel: satu untuk autentikasi pengguna dan satu untuk data utama sesuai tema. Tabel data utama memiliki minimal 5 kolom dengan tipe data yang bervariasi.
- Autentikasi dan *Session*: terdapat fitur registrasi, *login*, dan *logout*. *Password* disimpan dalam bentuk *hash*, dan sesi pengguna dikelola dengan *session*
- Operasi *CRUD* pada Halaman Terproteksi: implementasikan keempat operasi *Create*, *Read*, *Update*, dan *Delete* pada data utama. Seluruh halaman *CRUD* hanya dapat diakses setelah pengguna *login*, dan terapkan minimal 2 role (admin dan user) dengan kewenangan yang berbeda.
- Keamanan dan Struktur Kode: gunakan prepared statement pada seluruh query yang melibatkan input pengguna, gunakan `htmlspecialchars()` saat menampilkan data ke HTML, serta pisahkan file koneksi dan file pengecekan autentikasi dari halaman lain.
- Tampilan dan Validasi: gunakan framework CSS (*Bootstrap* atau *TailwindCSS*) untuk mempercantik tampilan, dan tambahkan validasi *form* sederhana di sisi browser

4.2 Source Code

4.2.1 Source Code Soal No. 1

Membuat tabel

```
CREATE TABLE users (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nama VARCHAR(100) NOT NULL,  
  username VARCHAR(100) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  role ENUM('admin','user') NOT NULL DEFAULT 'user'  
);  
  
CREATE TABLE buku (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  judul VARCHAR(200) NOT NULL,  
  penulis VARCHAR(100) NOT NULL,
```

```

    penerbit VARCHAR(100) NOT NULL,
    tahun_terbit INT NOT NULL,
    stok INT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Register.php

```

<?php
include 'config/koneksi.php';

$errorNama = "";
$errorUsername = "";
$errorPassword = "";
$success = "";

$nama = "";
$username = "";

if (isset($_POST['register'])) {

    $nama = trim($_POST['nama']);
    $username = trim($_POST['username']);
    $passwordInput = $_POST['password'];

    $role = "user";

    if ($nama == "") {
        $errorNama = "Nama wajib diisi";
    }

    // validasi password
    if (strlen($passwordInput) < 6) {
        $errorPassword = "Password minimal 6 karakter";
    }

    $cek = $conn->prepare(
        "SELECT id FROM users WHERE username = ?"
    );

    $cek->bind_param("s", $username);
    $cek->execute();

    $result = $cek->get_result();

    if ($result->num_rows > 0) {
        $errorUsername = "Username sudah digunakan";
    }
}

```

```

    }

    if (
        empty($errorNama) &&
        empty($errorUsername) &&
        empty($errorPassword)
    ) {

        $password = password_hash(
            $passwordInput,
            PASSWORD_DEFAULT
        );

        $query = $conn->prepare(
            "INSERT INTO users (nama, username, password, role)
            VALUES (?, ?, ?, ?)"
        );

        $query->bind_param(
            "ssss",
            $nama,
            $username,
            $password,
            $role
        );

        if ($query->execute()) {

            // redirect supaya form kosong saat refresh
            header("Location: register.php?success=1");
            exit;

        } else {
            $errorUsername = "Register gagal";
        }
    }
}
?>

<!DOCTYPE html>
<html>

<head>
    <title>Register</title>

    <script src="https://cdn.tailwindcss.com"></script>
</head>

```

```

<body class="bg-gray-100 flex justify-center items-center h-screen">

    <div class="bg-white p-8 rounded shadow w-96">

        <h1 class="text-2xl font-bold mb-4 text-center">
            Register
        </h1>

        <!-- notif sukses -->
        <?php if (isset($_GET['success'])) : ?>

            <div class="bg-green-100 text-green-700 p-2 rounded
mb-3 text-sm">
                Register berhasil
            </div>

        <?php endif; ?>

        <form method="POST">

            <div class="mb-3">

                <input
                    type="text"
                    name="nama"
                    placeholder="Nama"
                    class="w-full border p-2 rounded"
                    value="<?php echo htmlspecialchars($nama); ?>"
                    required>

                <?php if (!empty($errorNama)) : ?>

                    <p class="text-red-500 text-sm mt-1">
                        <?php echo $errorNama; ?>
                    </p>

                <?php endif; ?>

            </div>

            <div class="mb-3">

                <input
                    type="text"
                    name="username"

```

```

        placeholder="Username"
        class="w-full border p-2 rounded"
        value="<?php echo htmlspecialchars($username);
?>"

        required>

        <?php if (!empty($errorUsername)) : ?>

            <p class="text-red-500 text-sm mt-1">
                <?php echo $errorUsername; ?>
            </p>

        <?php endif; ?>

    </div>

    <div class="mb-3">

        <input
            type="password"
            name="password"
            placeholder="Password"
            class="w-full border p-2 rounded"

            required>

        <?php if (!empty($errorPassword)) : ?>

            <p class="text-red-500 text-sm mt-1">
                <?php echo $errorPassword; ?>
            </p>

        <?php endif; ?>

    </div>

    <button
        type="submit"
        name="register"
        class="bg-blue-500 hover:bg-blue-600 text-white w-
full p-2 rounded">

        Register

    </button>

</form>

```

```

        <p class="mt-3 text-center text-sm">
            Sudah punya akun?

            <a href="login.php" class="text-blue-500">
                Login
            </a>
        </p>

    </div>

</body>

</html>

```

Login.php

```

<?php
session_start();
include 'config/koneksi.php';

if (isset($_POST['login'])) {

    $username = $_POST['username'];
    $password = $_POST['password'];

    $query = $conn->prepare("SELECT * FROM users WHERE username = ?");
    $query->bind_param("s", $username);
    $query->execute();

    $result = $query->get_result();

    if ($result->num_rows > 0) {

        $data = $result->fetch_assoc();

        if (password_verify($password, $data['password'])) {

            $_SESSION['login'] = true;
            $_SESSION['id'] = $data['id'];
            $_SESSION['nama'] = $data['nama'];
            $_SESSION['role'] = $data['role'];

            header("Location: dashboard.php");
            exit;
        } else {

```



```

        $error = "Password salah";
    }
} else {
    $error = "Username tidak ditemukan";
}
}
?>

<!DOCTYPE html>
<html>

<head>
    <title>Login</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100 flex justify-center items-center h-screen">

    <div class="bg-white p-8 rounded shadow w-96">

        <h1 class="text-2xl font-bold mb-4 text-center">Login</h1>

        <?php if (isset($error)) : ?>
            <p class="text-red-500 text-sm mb-3"><?php echo
$error; ?></p>
        <?php endif; ?>

        <?php if (isset($success)) : ?>
            <p class="text-green-500 text-sm mb-3"><?php echo
$success; ?></p>
        <?php endif; ?>

        <form method="POST">

            <input type="text" name="username"
placeholder="Username"
                class="w-full border p-2 mb-3 rounded" required>

            <input type="password" name="password"
placeholder="Password"
                class="w-full border p-2 mb-3 rounded" required>

            <button type="submit" name="login"
                class="bg-green-500 text-white w-full p-2
rounded">
                Login
    
```

```

        </button>

    </form>

    <p class="mt-3 text-center">
        Belum punya akun?
        <a href="register.php" class="text-blue-500">Register</a>
    </p>

</div>

</body>

</html>

```

Logout.php

```

php
<?php
session_start();
session_destroy();

header("Location: login.php");
exit;
?>

```

Dashboard.php

```

php
<?php
include 'config/auth.php';
include 'config/koneksi.php';

$data = $conn->query("SELECT * FROM buku");
?>

<!DOCTYPE html>
<html>

<head>
    <title>Dashboard</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100">

    <div class="container mx-auto p-5">

```

```

        <div class="flex justify-between items-center mb-5">
            <div>
                <h1 class="text-3xl font-bold">Sistem
Perpustakaan</h1>
                <p>Selamat datang, <?php echo
htmlspecialchars($_SESSION['nama']); ?></p>
            </div>

            <a href="logout.php"
                class="bg-red-500 text-white px-4 py-2 rounded">
                Logout
            </a>
        </div>

        <?php if ($_SESSION['role'] == 'admin') : ?>

            <a href="tambah_buku.php"
                class="bg-blue-500 text-white px-4 py-2 rounded
inline-block mb-4">
                Tambah Buku
            </a>

        <?php endif; ?>

        <div class="bg-white p-5 rounded shadow">

            <table class="w-full border">
                <tr class="bg-gray-200">
                    <th class="border p-2">No</th>
                    <th class="border p-2">Judul</th>
                    <th class="border p-2">Penulis</th>
                    <th class="border p-2">Penerbit</th>
                    <th class="border p-2">Tahun</th>
                    <th class="border p-2">Stok</th>
                    <th class="border p-2">Aksi</th>
                </tr>

                <?php
                $no = 1;
                while ($row = $data->fetch_assoc()) :
                ?>

                    <tr>
                        <td class="border p-2"><?php echo $no++;
?></td>

```

```

        <td class="border p-2"><?php echo
htmlspecialchars($row['judul']); ?></td>
        <td class="border p-2"><?php echo
htmlspecialchars($row['penulis']); ?></td>
        <td class="border p-2"><?php echo
htmlspecialchars($row['penerbit']); ?></td>
        <td class="border p-2"><?php echo
htmlspecialchars($row['tahun_terbit']); ?></td>
        <td class="border p-2"><?php echo
htmlspecialchars($row['stok']); ?></td>

        <td class="border p-2">

            <?php if ($_SESSION['role'] ==
'admin') : ?>

                <a href="edit_buku.php?id=<?php
echo $row['id']; ?>"
                    class="bg-yellow-400 px-3 py-1
rounded text-white mr-4">
                    Edit
                </a>

                <a href="hapus_buku.php?id=<?php
echo $row['id']; ?>"
                    onclick="return confirm('Yakin
hapus data?')"
                    class="bg-red-500 px-3 py-1
rounded text-white">
                    Hapus
                </a>

            <?php else : ?>

                <span class="text-gray-500">Hanya
lihat</span>

            <?php endif; ?>

        </td>
    </tr>

    <?php endwhile; ?>

</table>

</div>

```

```
        </div>

</body>

</html>
```

Auth.php

```
<?php
session_start();

if (!isset($_SESSION['login'])) {
    header("Location: login.php");
    exit;
}
```

Koneksi.php

```
<?php

$host = "localhost";
$user = "root";
$pass = "";
$db = "perpustakaan_db";

$conn = new mysqli($host, $user, $pass, $db);

if ($conn->connect_error) {
    die("Koneksi gagal: " . $conn->connect_error);
}
```

Tambah_buku.php

```
<?php
include 'config/auth.php';
include 'config/koneksi.php';

if ($_SESSION['role'] != 'admin') {
    header("Location: login.php");
    exit;
}

$errorTahun = "";
$errorStok = "";

$judul = "";
$penulis = "";
```

```

$penerbit = "";
$tahun = "";
$stok = "";

if (isset($_POST['simpan'])) {

    $judul = trim($_POST['judul']);
    $penulis = trim($_POST['penulis']);
    $penerbit = trim($_POST['penerbit']);
    $tahun = $_POST['tahun'];
    $stok = $_POST['stok'];

    if ($tahun <= 0) {

        $errorTahun = "Tahun tidak boleh 0";
    } elseif ($tahun < 1900) {

        $errorTahun = "Tahun minimal 1900";
    }

    if ($stok < 0) {

        $errorStok = "Stok tidak boleh minus";
    }

    if (
        empty($errorTahun) &&
        empty($errorStok)
    ) {

        $query = $conn->prepare(
            "INSERT INTO buku
            (judul, penulis, penerbit, tahun_terbit, stok)
            VALUES (?, ?, ?, ?, ?)"
        );

        $query->bind_param(
            "sssi",
            $judul,
            $penulis,
            $penerbit,
            $tahun,
            $stok
        );

        if ($query->execute()) {

```

```

        header("Location: dashboard.php");
        exit;
    }
}
?>

<!DOCTYPE html>
<html>

<head>
    <title>Tambah Buku</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100">

    <div class="container mx-auto p-5">

        <div class="bg-white p-5 rounded shadow max-w-lg mx-auto">

            <h1 class="text-2xl font-bold mb-4">
                Tambah Buku
            </h1>

            <form method="POST">

                <!-- judul -->
                <div class="mb-3">

                    <input
                        type="text"
                        name="judul"
                        placeholder="Judul Buku"
                        class="w-full border p-2 rounded"

                        value="<?php echo
htmlspecialchars($judul); ?>"

                        required>

                    </div>

                <!-- penulis -->
                <div class="mb-3">

                    <input

```

```

        type="text"
        name="penulis"
        placeholder="Penulis"
        class="w-full border p-2 rounded"

        value="<?php echo
htmlspecialchars($penulis); ?>"

        required>

</div>

<!-- penerbit -->
<div class="mb-3">

    <input
        type="text"
        name="penerbit"
        placeholder="Penerbit"
        class="w-full border p-2 rounded"

        value="<?php echo
htmlspecialchars($penerbit); ?>"

        required>

</div>

<div class="mb-3">

    <input
        type="number"
        name="tahun"
        placeholder="Tahun Terbit"
        class="w-full border p-2 rounded"

        value="<?php echo
htmlspecialchars($tahun); ?>"

        required>

    <?php if (!empty($errorTahun)) : ?>

        <p class="text-red-500 text-sm mt-1">
            <?php echo $errorTahun; ?>
        </p>

```



```

        <?php endif; ?>

    </div>

    <div class="mb-3">

        <input
            type="number"
            name="stok"
            placeholder="Stok"
            class="w-full border p-2 rounded"

            value="<?php echo htmlspecialchars($stok);
?>"

            required>

        <?php if (!empty($errorStok)) : ?>

            <p class="text-red-500 text-sm mt-1">
                <?php echo $errorStok; ?>
            </p>

        <?php endif; ?>

    </div>

    <button
        type="submit"
        name="simpan"
        class="bg-blue-500 text-white px-4 py-2
rounded">

        Simpan

    </button>

</form>

</div>

</div>

</body>

</html>

```

Edit_buku.php

```
<?php
include 'config/auth.php';
include 'config/koneksi.php';

if ($_SESSION['role'] != 'admin') {
    header("Location: login.php");
    exit;
}

$id = $_GET['id'];

$query = $conn->prepare(
    "SELECT * FROM buku WHERE id = ?"
);

$query->bind_param("i", $id);
$query->execute();

$result = $query->get_result();
$buku = $result->fetch_assoc();

$errorTahun = "";
$errorStok = "";

if (isset($_POST['update'])) {

    $judul = trim($_POST['judul']);
    $penulis = trim($_POST['penulis']);
    $penerbit = trim($_POST['penerbit']);
    $tahun = $_POST['tahun'];
    $stok = $_POST['stok'];

    if ($tahun <= 0) {

        $errorTahun = "Tahun tidak boleh 0";
    } elseif ($tahun < 1900) {

        $errorTahun = "Tahun minimal 1900";
    }

    if ($stok < 0) {

        $errorStok = "Stok tidak boleh minus";
    }
}
```

```

if (
    empty($errorTahun) &&
    empty($errorStok)
) {

    $update = $conn->prepare(
        "UPDATE buku SET
        judul = ?,
        penulis = ?,
        penerbit = ?,
        tahun_terbit = ?,
        stok = ?
        WHERE id = ?"
    );

    $update->bind_param(
        "sssi",
        $judul,
        $penulis,
        $penerbit,
        $tahun,
        $stok,
        $id
    );

    if ($update->execute()) {

        header("Location: dashboard.php");
        exit;
    }
}
?>

<!DOCTYPE html>
<html>

<head>
    <title>Edit Buku</title>
    <script src="https://cdn.tailwindcss.com"></script>
</head>

<body class="bg-gray-100">

    <div class="container mx-auto p-5">

        <div class="bg-white p-5 rounded shadow max-w-lg mx-auto">

```

```

<h1 class="text-2xl font-bold mb-4">
    Edit Buku
</h1>

<form method="POST">

    <!-- judul -->
    <div class="mb-3">

        <input
            type="text"
            name="judul"
            class="w-full border p-2 rounded"

            value="<?php
                echo htmlspecialchars(
                    isset($_POST['judul'])
                    ? $_POST['judul']
                    : $buku['judul']
                );
                ?>"

            required>

    </div>

    <div class="mb-3">

        <input
            type="text"
            name="penulis"
            class="w-full border p-2 rounded"

            value="<?php
                echo htmlspecialchars(
                    isset($_POST['penulis'])
                    ? $_POST['penulis']
                    : $buku['penulis']
                );
                ?>"

            required>

    </div>

    <div class="mb-3">

```

```

        <input
            type="text"
            name="penerbit"
            class="w-full border p-2 rounded"

            value="<?php
                echo htmlspecialchars(
                    isset($_POST['penerbit'])
                        ? $_POST['penerbit']
                        : $buku['penerbit']
                );
            ?>"

            required>

    </div>

    <!-- tahun -->
    <div class="mb-3">

        <input
            type="number"
            name="tahun"
            class="w-full border p-2 rounded"

            value="<?php
                echo htmlspecialchars(
                    isset($_POST['tahun'])
                        ? $_POST['tahun']
                        : $buku['tahun_terbit']
                );
            ?>"

            required>

        <?php if (!empty($errorTahun)) : ?>

            <p class="text-red-500 text-sm mt-1">
                <?php echo $errorTahun; ?>
            </p>

        <?php endif; ?>

    </div>

    <!-- stok -->

```

```

<div class="mb-3">

    <input
        type="number"
        name="stok"
        class="w-full border p-2 rounded"

        value="<?php
            echo htmlspecialchars(
                isset($_POST['stok'])
                ? $_POST['stok']
                : $buku['stok']
            );
            ?>"

        required>

    <?php if (!empty($errorStok)) : ?>

        <p class="text-red-500 text-sm mt-1">
            <?php echo $errorStok; ?>
        </p>

    <?php endif; ?>

</div>

<button
    type="submit"
    name="update"
    class="bg-yellow-500 text-white px-4 py-2
rounded">

    Update

</button>

</form>

</div>

</div>

</body>

</html>

```

Hapus_buku.php

```
<?php
include 'config/auth.php';
include 'config/koneksi.php';

if ($_SESSION['role'] != 'admin') {
    header("Location: login.php");
    exit;
}

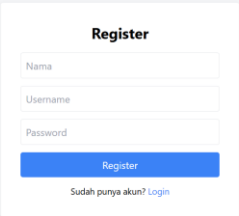
$id = $_GET['id'];

$query = $conn->prepare("DELETE FROM buku WHERE id = ?");
$query->bind_param("i", $id);
$query->execute();

header("Location: dashboard.php");
exit;
```

4.3 Hasil

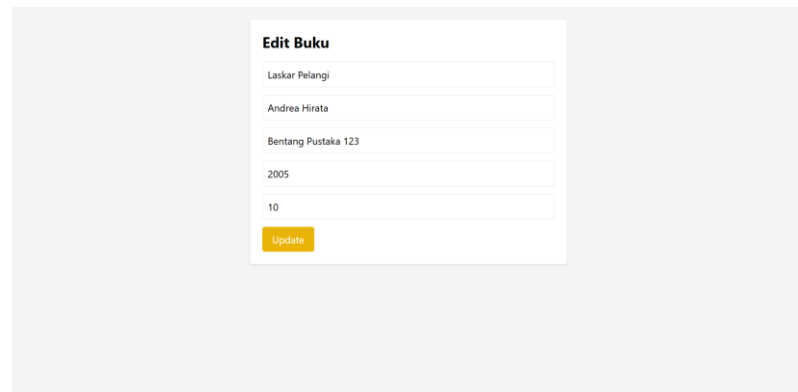
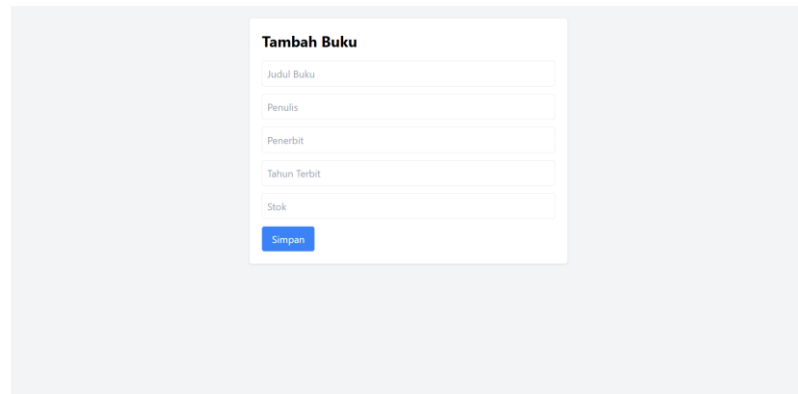
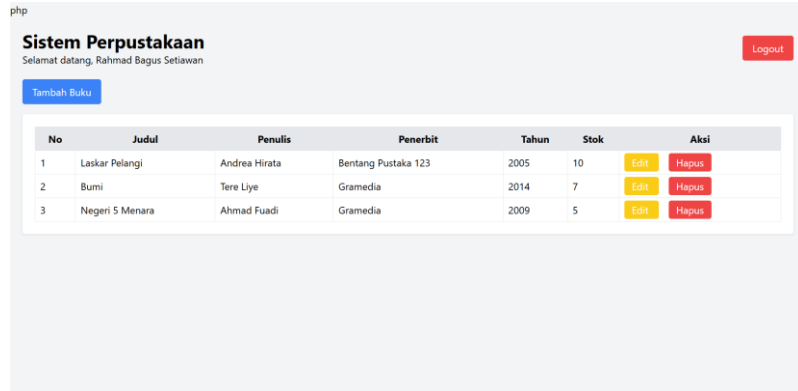
4.3.1 Hasil Soal No. 1



The screenshot shows a 'Register' form with a title 'Register' in bold. It contains three input fields labeled 'Nama', 'Username', and 'Password'. Below these fields is a blue 'Register' button. At the bottom, there is a link that says 'Sudah punya akun? Login'.



The screenshot shows a 'Login' form with a title 'Login' in bold. It contains two input fields labeled 'Username' and 'Password'. Below these fields is a green 'Login' button. At the bottom, there is a link that says 'Belum punya akun? Register'. There is also a red error message above the Username field that says 'Username tidak ditemukan'.



4.4 Penjelasan

4.4.1 Penjelasan Soal No. 1

Sistem perpustakaan ini merupakan aplikasi web sederhana berbasis PHP dan MySQL yang dibuat untuk mengelola data buku dengan fitur autentikasi pengguna dan *CRUD*. Sistem memiliki dua *role* yaitu admin dan *user*, dimana admin dapat menambah, mengedit, menghapus, dan melihat data buku, sedangkan user hanya dapat melihat data buku. Data pengguna disimpan pada tabel *users* dan data buku disimpan pada tabel *buku*. Sistem menggunakan *session* untuk menyimpan status login, *password_hash()* dan *password_verify()* untuk keamanan *password*, *prepared statement* untuk mencegah *SQL Injection*, serta

`htmlspecialchars()` untuk mengamankan *output* dari serangan *XSS*. Tampilan dibuat menggunakan *TailwindCSS* agar lebih modern dan responsif, serta dilengkapi validasi *form* sederhana agar *input* pengguna lebih terkontrol.

BAB V

PENUTUP

5.1 Analisa

Perkembangan teknologi informasi mendorong kebutuhan akan aplikasi berbasis web yang mampu mengelola data secara dinamis dan persisten. Dalam konteks pengembangan sistem informasi, kemampuan membangun antarmuka pengguna yang terhubung langsung dengan basis data merupakan kompetensi dasar yang wajib dikuasai. *HTML Form* berperan sebagai jembatan antara pengguna dan server, memungkinkan pengiriman data secara terstruktur melalui method *GET* maupun *POST*. Di sisi server, *PHP* menjadi bahasa pemrograman yang memproses data tersebut sebelum berinteraksi dengan *MySQL* sebagai sistem manajemen basis data. Namun, seiring meningkatnya kompleksitas aplikasi, aspek keamanan seperti *SQL Injection* dan *XSS* tidak dapat diabaikan. Oleh karena itu, pemahaman menyeluruh mengenai *HTML Form*, koneksi *PHP-MySQL*, operasi *CRUD*, serta mekanisme autentikasi dan otorisasi menjadi fondasi penting dalam pengembangan aplikasi web yang fungsional sekaligus aman. Sistem *autentikasi* memanfaatkan *session PHP* untuk menjaga status login lintas halaman, dengan *password_hash* dan *password_verify* memastikan kredensial tidak tersimpan dalam bentuk *plaintext*. Penerapan *role-based access control* membedakan kewenangan antara admin dan *user*, dengan proteksi berlapis pada sisi UI maupun server.

5.2 Kesimpulan

Modul ini mencakup alur kerja lengkap sebuah aplikasi web dinamis, mulai dari pengambilan input melalui *HTML Form* hingga penyimpanan dan pengelolaan data di database. Method *POST* dipilih untuk operasi yang memodifikasi data karena tidak mengekspos data sensitif di URL.

- a) *HTML Form* dan pemilihan method yang tepat merupakan fondasi utama dalam pengumpulan dan pengiriman data dari sisi pengguna ke server.
- b) Pemisahan file koneksi, autentikasi, dan logika *CRUD* ke dalam file tersendiri meningkatkan keterbacaan kode sekaligus mempermudah pemeliharaan aplikasi dalam jangka panjang.
- c) Penerapan prepared statement, hashing *password*, *session management*, dan *role-based access control* menjadikan aplikasi tidak hanya fungsional, tetapi juga memenuhi standar keamanan aplikasi web modern.